

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)

Applied Business Analytics Practical

Module - I

Practical – V

Roll No.:T032	Name:KHUSHI
Class:TYIT	Batch:2
Date of Assignment: 21/08/26	Date/Time of Submission: 21/08/26

5. Probability and Probability Distributions:

a. Simulate a probability experiment (e.g., rolling dice) using programming and calculate the probabilities of different outcomes.

CODE:-

A) DICE PROBABILITY EXPERIMENT

```
import numpy as np
import matplotlib.pyplot as plt
from statistics import mode
```

1. Simulate 1,000 dice rolls

```
np.random.seed(42)
dice = np.random.randint(1, 7, 1000)
print("\nFirst 20 dice outcomes:")
print(dice[:20])
```

Frequency of each outcome

```
frequency = np.bincount(dice)[1:]
```

2. Experimental Probability

```
experimental_probability = frequency / 1000
print("\nExperimental Probability:")
for number in range(1, 7):
    print(number, ":", experimental_probability[number - 1])
```

3. Theoretical Probability

```
theoretical_probability = 1 / 6
print("\nTheoretical Probability:")
for number in range(1, 7):
    print(number, ":", theoretical_probability)
```

4. Probability of getting 1,2,3,4,5,6

```
print("\nProbability of each number:")
for number in range(1, 7):
    print(f"P({number}) = {experimental_probability[number-1]:.3f}")
```

5. Probability of even number

```
even_probability = np.mean(dice % 2 == 0)
print("\nProbability of Even Number:", even_probability)
```

6. Probability of odd number

```
odd_probability = np.mean(dice % 2 != 0)
print("Probability of Odd Number:", odd_probability)
```

7. Probability greater than 4

```
greater_4 = np.mean(dice > 4)
print("Probability of Number > 4:", greater_4)
```

8. Probability less than 4

```
less_4 = np.mean(dice < 4)
print("Probability of Number < 4:", less_4)
```

9. Probability greater than or equal to 4

```
greater_equal_4 = np.mean(dice >= 4)
print("Probability of Number >= 4:", greater_equal_4)
```

10. Bar Chart - Frequency

```
plt.figure(figsize=(7, 5))
plt.bar(range(1, 7), frequency)
plt.xlabel("Dice Outcome")
plt.ylabel("Frequency")
plt.title("Frequency of Dice Outcomes")
plt.xticks(range(1, 7))
plt.show()
```

11. Bar Chart - Experimental Probability

```
plt.figure(figsize=(7, 5))
plt.bar(range(1, 7), experimental_probability)
plt.xlabel("Dice Outcome")
plt.ylabel("Experimental Probability")
plt.title("Experimental Probability of Dice Outcomes")
plt.xticks(range(1, 7))
plt.show()
```

12. Pie Chart

```
plt.figure(figsize=(7, 7))
plt.pie(frequency, labels=range(1, 7), autopct='%1.1f%%')
plt.title("Percentage Distribution of Dice Outcomes")
plt.show()
```

13. Experimental vs Theoretical Probability

```
plt.figure(figsize=(8, 5))
plt.plot(range(1, 7), experimental_probability,
         marker='o', label="Experimental")
plt.plot(range(1, 7), [theoretical_probability] * 6,
         marker='o', label="Theoretical")
plt.xlabel("Dice Outcome")
plt.ylabel("Probability")
plt.title("Experimental vs Theoretical Probability")
plt.xticks(range(1, 7))
plt.legend()
plt.grid()
plt.show()
```

14. Even vs Odd Frequency

```
even_frequency = np.sum(dice % 2 == 0)
odd_frequency = np.sum(dice % 2 != 0)
plt.figure(figsize=(6, 5))
plt.bar(["Even", "Odd"], [even_frequency, odd_frequency])
```

```
plt.xlabel("Type")
plt.ylabel("Frequency")
plt.title("Frequency of Even vs Odd Outcomes")
plt.show()
```

15. Experimental Probability Chart

```
probabilities = [
    even_probability,
    odd_probability,
    greater_4,
    less_4
]
labels = [
    "Even",
    "Odd",
    "> 4",
    "< 4"
]
plt.figure(figsize=(7, 5))
plt.bar(labels, probabilities)
plt.xlabel("Event")
plt.ylabel("Experimental Probability")
plt.title("Experimental Probability of Different Events")
plt.show()
print("KHUSHI T032")
```

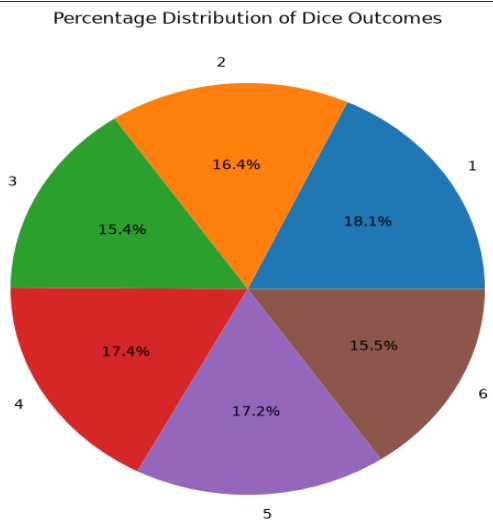
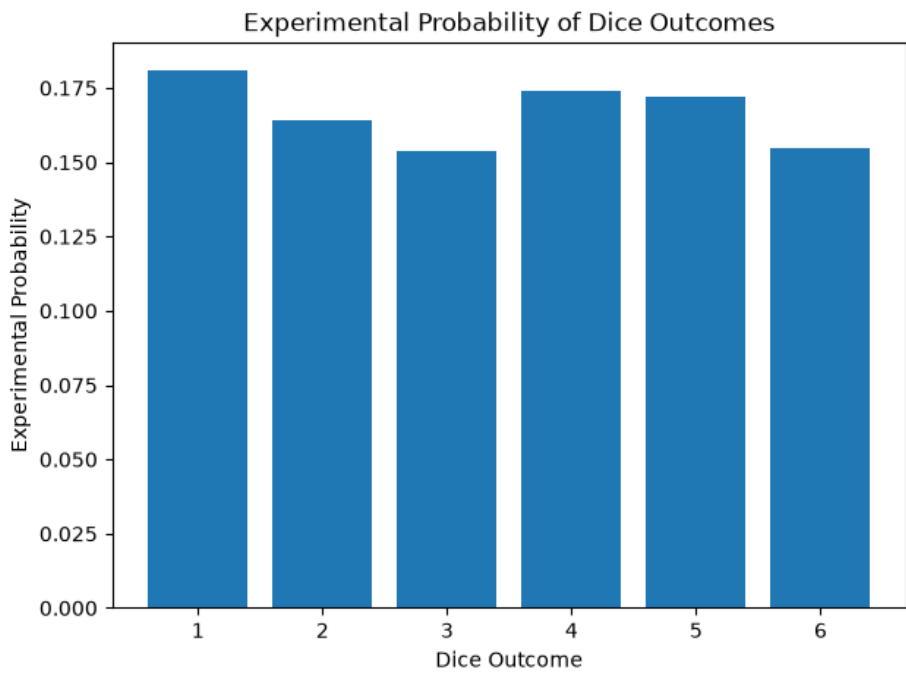
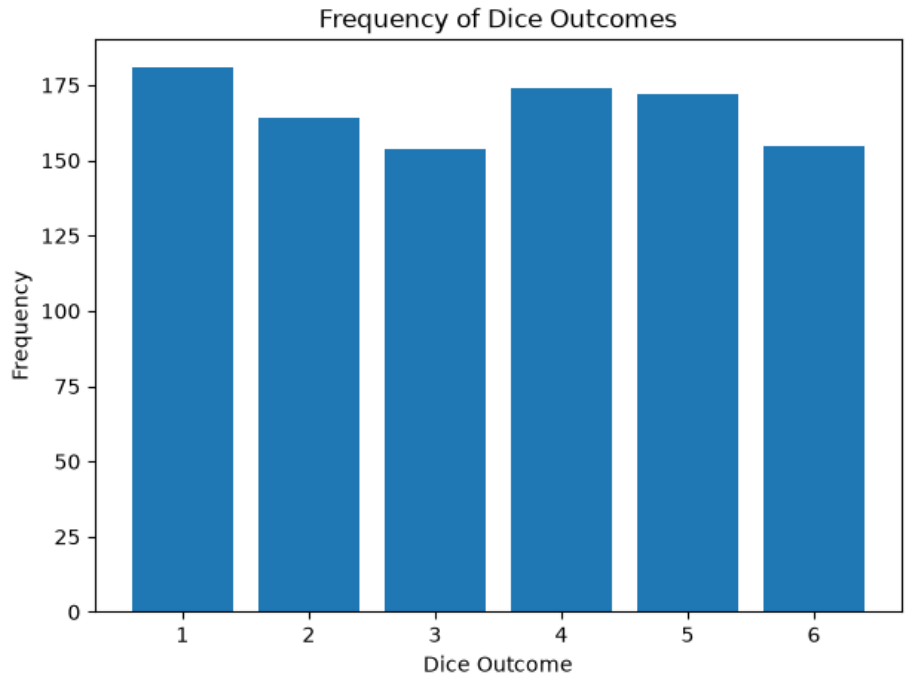
OUTPUT:-

```
First 20 dice outcomes:
[4 5 3 5 5 2 3 3 3 5 4 3 6 5 2 4 6 6 2 4]

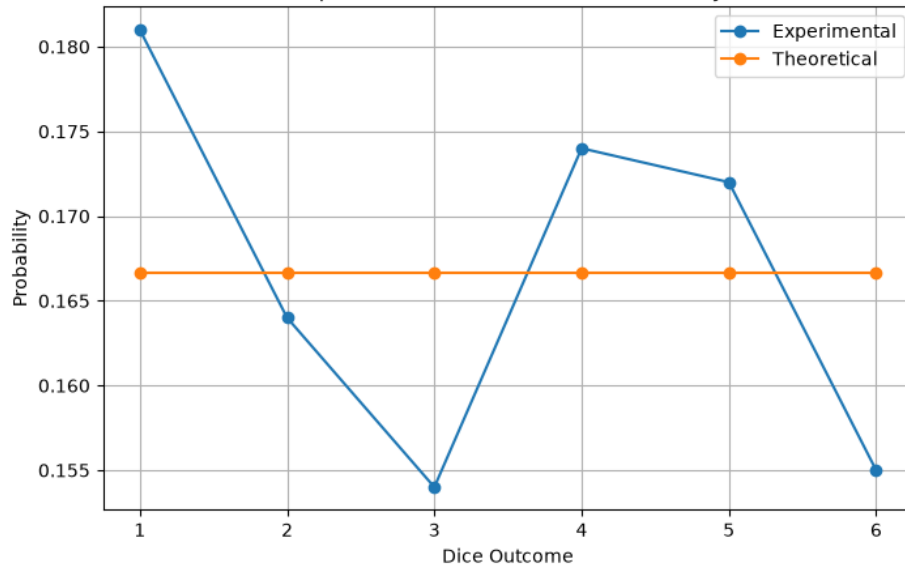
Experimental Probability:
1 : 0.181
2 : 0.164
3 : 0.154
4 : 0.174
5 : 0.172
6 : 0.155

Theoretical Probability:
1 : 0.16666666666666666
2 : 0.16666666666666666
3 : 0.16666666666666666
4 : 0.16666666666666666
5 : 0.16666666666666666
6 : 0.16666666666666666

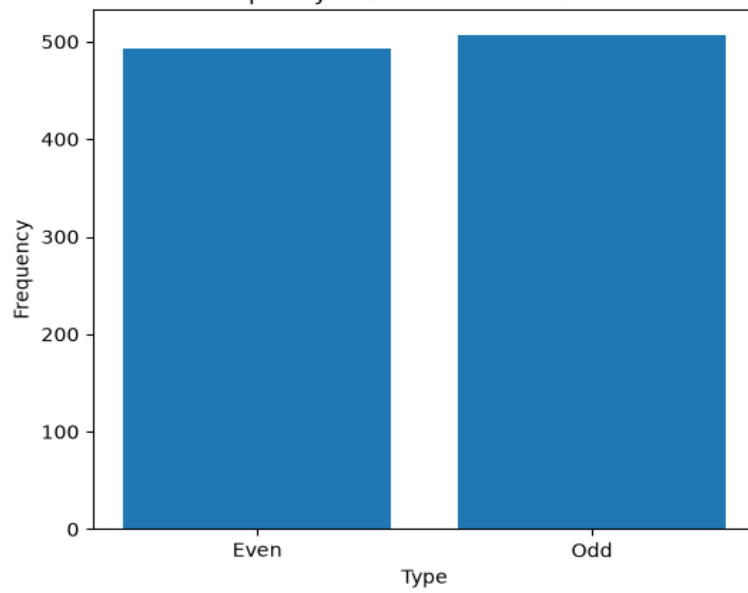
Probability of each number:
P(1) = 0.181
P(2) = 0.164
P(3) = 0.154
P(4) = 0.174
...
Probability of Odd Number: 0.507
Probability of Number > 4: 0.327
Probability of Number < 4: 0.499
Probability of Number >= 4: 0.501
```



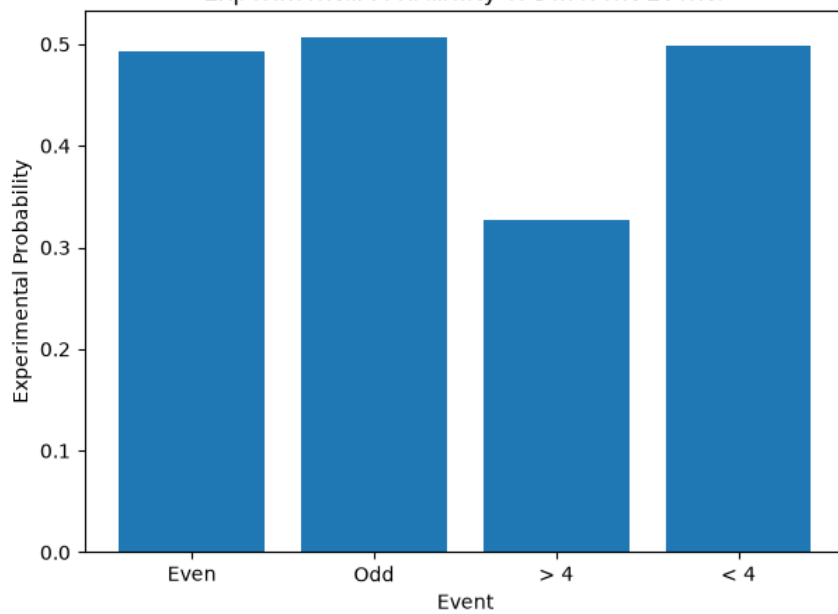
Experimental vs Theoretical Probability



Frequency of Even vs Odd Outcomes



Experimental Probability of Different Events



b. Generate random numbers from various probability distributions (normal, uniform, exponential) and analyze their properties.

CODE:-

1. NORMAL DISTRIBUTION

```
print("\n----- Normal Distribution -----")
normal_data = np.random.normal(
    loc=50,
    scale=10,
    size=1000
)
```

Statistics

```
normal_mean = np.mean(normal_data)
normal_median = np.median(normal_data)
normal_mode = mode(np.round(normal_data, 1))
normal_variance = np.var(normal_data)
normal_std = np.std(normal_data)
normal_min = np.min(normal_data)
normal_max = np.max(normal_data)
```

```
print("Mean:", normal_mean)
print("Median:", normal_median)
print("Mode:", normal_mode)
print("Variance:", normal_variance)
print("Standard Deviation:", normal_std)
print("Minimum:", normal_min)
print("Maximum:", normal_max)
```

4. Histogram

```
plt.figure(figsize=(7, 5))
plt.hist(normal_data, bins=30)
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.title("Histogram of Normal Distribution")
plt.show()
```

5. Box Plot

```
plt.figure(figsize=(7, 5))
plt.boxplot(normal_data)
plt.ylabel("Value")
plt.title("Box Plot of Normal Distribution")
plt.show()
```

6. Percentage between 40 and 60

```
percentage_40_60 = np.mean(
    (normal_data >= 40) & (normal_data <= 60)
) * 100
```

```
print("Percentage between 40 and 60:",
      percentage_40_60, "%")
```

7. Percentage between 30 and 70

```
percentage_30_70 = np.mean(
    (normal_data >= 30) & (normal_data <= 70)
) * 100
```

```
print("Percentage between 30 and 70:",  
      percentage_30_70, "%")
```

8. UNIFORM DISTRIBUTION

```
print("\n----- Uniform Distribution -----")
```

```
uniform_data = np.random.uniform(  
    0,  
    100,  
    1000  
)
```

```
print("Mean:", np.mean(uniform_data))  
print("Minimum:", np.min(uniform_data))  
print("Maximum:", np.max(uniform_data))
```

9. Histogram

```
plt.figure(figsize=(7, 5))  
plt.hist(uniform_data, bins=30)  
plt.xlabel("Value")  
plt.ylabel("Frequency")  
plt.title("Histogram of Uniform Distribution")  
plt.show()
```

10. Box Plot

```
plt.figure(figsize=(7, 5))  
plt.boxplot(uniform_data)  
plt.ylabel("Value")  
plt.title("Box Plot of Uniform Distribution")  
plt.show()
```

11. EXPONENTIAL DISTRIBUTION

```
print("\n----- Exponential Distribution -----")
```

```
exponential_data = np.random.exponential(  
    scale=2,  
    size=1000  
)
```

```
print("Mean:", np.mean(exponential_data))  
print("Minimum:", np.min(exponential_data))  
print("Maximum:", np.max(exponential_data))
```

12. Histogram

```
plt.figure(figsize=(7, 5))  
plt.hist(exponential_data, bins=30)  
plt.xlabel("Value")  
plt.ylabel("Frequency")  
plt.title("Histogram of Exponential Distribution")  
plt.show()
```

13. Box Plot

```
plt.figure(figsize=(7, 5))  
plt.boxplot(exponential_data)
```

```

plt.ylabel("Value")
plt.title("Box Plot of Exponential Distribution")
plt.show()

# Possible extreme values
Q1 = np.percentile(exponential_data, 25)
Q3 = np.percentile(exponential_data, 75)
IQR = Q3 - Q1

upper_limit = Q3 + 1.5 * IQR

extreme_values = exponential_data[
    exponential_data > upper_limit
]

print("Possible Extreme Values:", len(extreme_values))

# 14. Percentage greater than 2
percentage_greater_2 = np.mean(
    exponential_data > 2
) * 100

print("Percentage greater than 2:",
      percentage_greater_2, "%")

# 15. Percentage greater than 5
percentage_greater_5 = np.mean(
    exponential_data > 5
) * 100

print("Percentage greater than 5:",
      percentage_greater_5, "%")
print("KHUSHI T032")

```

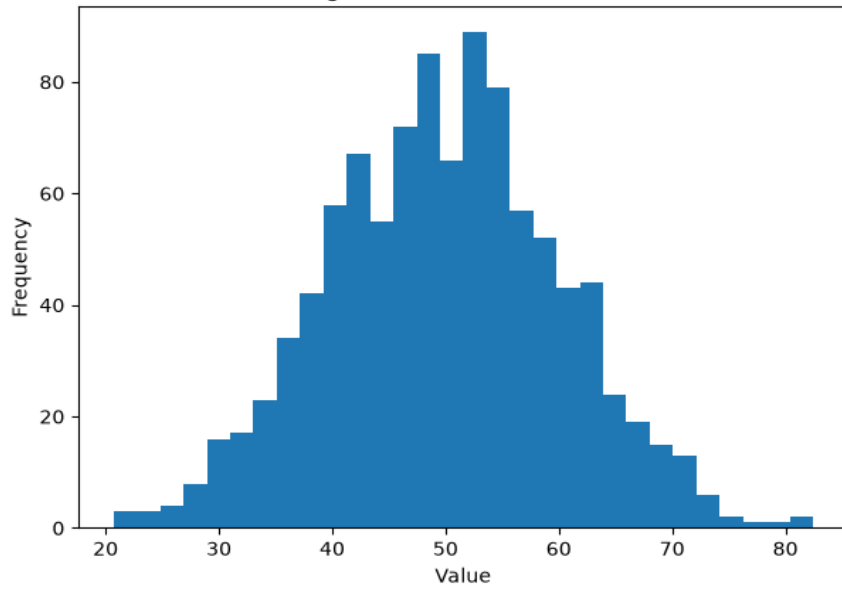
OUTPUT:-

```

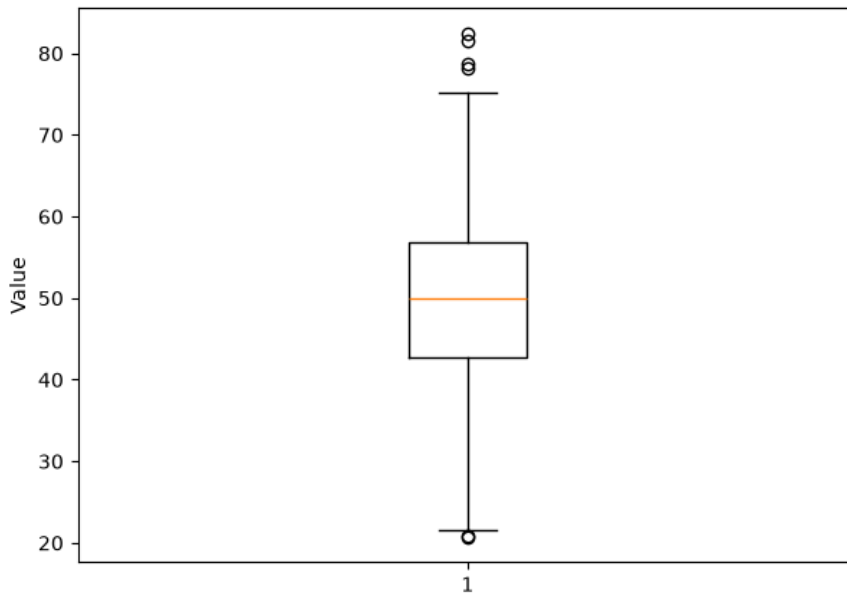
Mean: 49.866001972859436
Median: 49.98641243234685
Mode: 51.8
Variance: 104.46230413253552
Standard Deviation: 10.220680218680924
Minimum: 20.70551309997732
Maximum: 82.43092969594733

```

Histogram of Normal Distribution



Box Plot of Normal Distribution



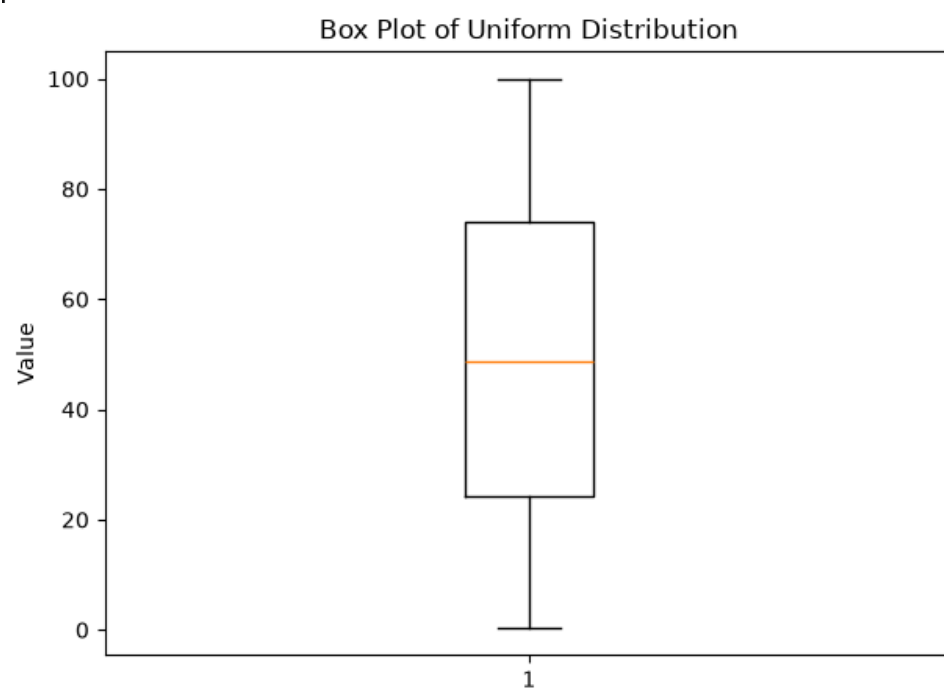
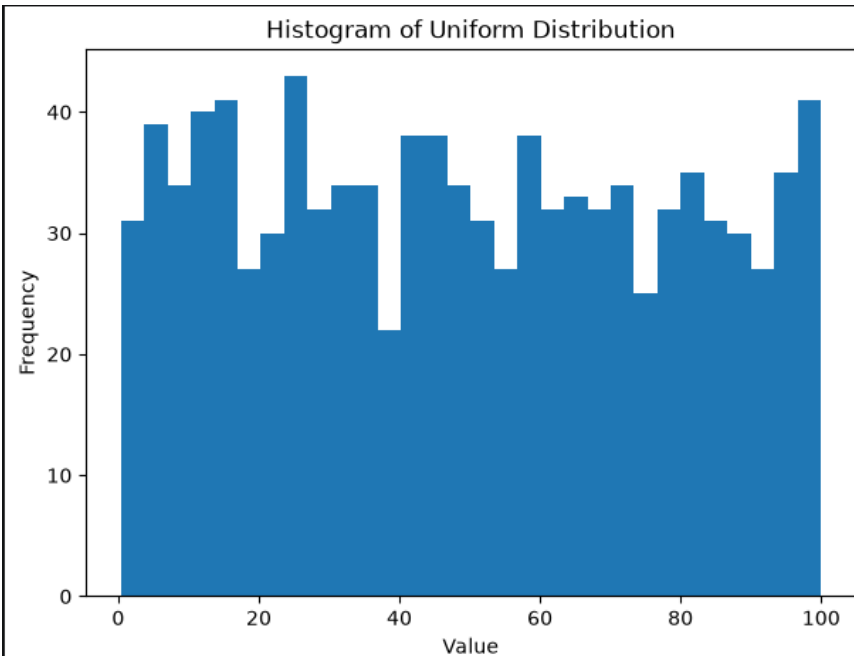
Percentage between 40 and 60: 66.2 %
Percentage between 30 and 70: 94.8 %

----- Uniform Distribution -----

Mean: 49.313457646243165

Minimum: 0.3918374951093262

Maximum: 99.94606810596731

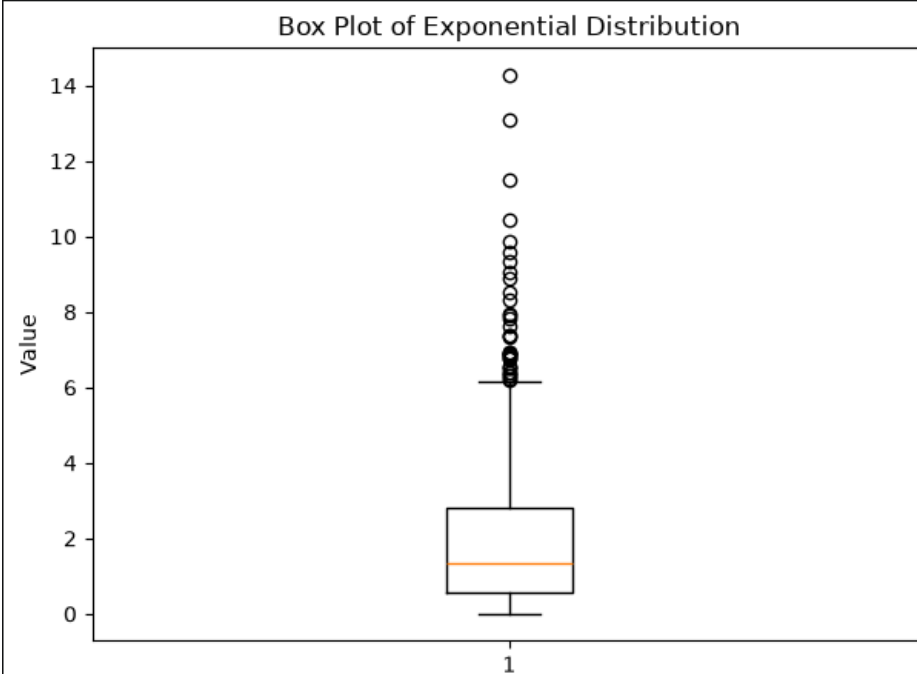
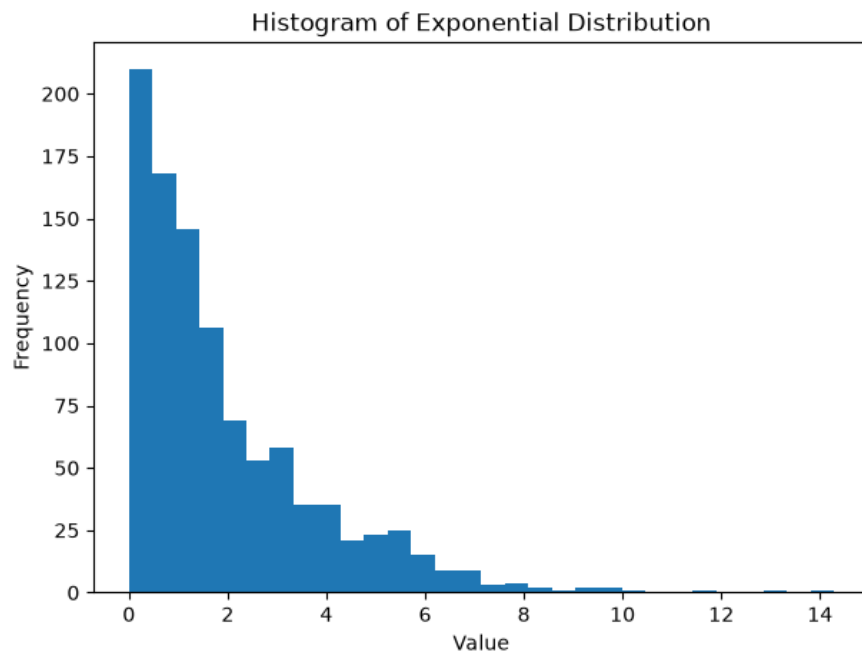


----- Exponential Distribution -----

Mean: 1.9658423961422273

Minimum: 0.0004820674628612675

Maximum: 14.280128770521504



```
Possible Extreme Values: 36
Percentage greater than 2: 35.3 %
Percentage greater than 5: 8.6 %
KHUSHI T032
```

C) COIN TOSS TASK

```
print("\n===== C) COIN TOSS =====")
```

1. Simulate 1,000 coin tosses

```
coin_1000 = np.random.choice(
    ["Heads", "Tails"],
    size=1000
)
```

```
heads_1000 = np.sum(coin_1000 == "Heads")
```

```
tails_1000 = np.sum(coin_1000 == "Tails")
```

```
experimental_heads = heads_1000 / 1000
```

```
experimental_tails = tails_1000 / 1000
```

```
print("\nTotal Tosses:", 1000)
```

```
print("Heads:", heads_1000)
```

```
print("Tails:", tails_1000)
```

```
print("Experimental Probability of Heads:",  
      experimental_heads)
```

```
print("Experimental Probability of Tails:",  
      experimental_tails)
```

2. Theoretical probability

```
print("\nTheoretical Probability:")
```

```
print("Heads:", 0.5)
```

```
print("Tails:", 0.5)
```

3. Bar Chart

```
plt.figure(figsize=(6, 5))
```

```
plt.bar(  
    ["Heads", "Tails"],  
    [heads_1000, tails_1000]  
)
```

```
plt.xlabel("Outcome")
```

```
plt.ylabel("Frequency")
```

```
plt.title("Frequency of Heads and Tails")
```

```
plt.show()
```

4. Pie Chart

```
plt.figure(figsize=(6, 6))
```

```
plt.pie(  
    [heads_1000, tails_1000],  
    labels=["Heads", "Tails"],  
    autopct='%1.1f%%'  
)
```

```
plt.title("Percentage of Heads and Tails")
```

```
plt.show()
```

5. Given example

```
heads = 8
```

```
total = 10
```

```
experimental_probability_heads = heads / total
```

```
print("\nIf coin is tossed 10 times and Heads occurs 8 times:")
```

```
print("Experimental Probability of Heads:",  
      experimental_probability_heads)
```

6. Simulate 10,000 coin tosses

```
print("\n----- 10,000 Coin Toss Analysis -----")
```

```

coin_10000 = np.random.choice(
    ["Heads", "Tails"],
    size=10000
)

heads_10000 = np.sum(coin_10000 == "Heads")
tails_10000 = np.sum(coin_10000 == "Tails")

exp_heads = heads_10000 / 10000
exp_tails = tails_10000 / 10000

theoretical_heads = 0.5
theoretical_tails = 0.5

difference_heads = abs(
    exp_heads - theoretical_heads
)

difference_tails = abs(
    exp_tails - theoretical_tails
)

print("\nTotal Tosses:", 10000)
print("Number of Heads:", heads_10000)
print("Number of Tails:", tails_10000)

print("Experimental Probability of Heads:",
      exp_heads)

print("Experimental Probability of Tails:",
      exp_tails)

print("Theoretical Probability of Heads:",
      theoretical_heads)

print("Theoretical Probability of Tails:",
      theoretical_tails)

print("Difference for Heads:",
      difference_heads)

print("Difference for Tails:",
      difference_tails)

# Bar Chart - 10,000 Tosses
plt.figure(figsize=(6, 5))
plt.bar(
    ["Heads", "Tails"],
    [heads_10000, tails_10000]
)
plt.xlabel("Outcome")
plt.ylabel("Frequency")
plt.title("10,000 Coin Tosses - Heads vs Tails")
plt.show()

```

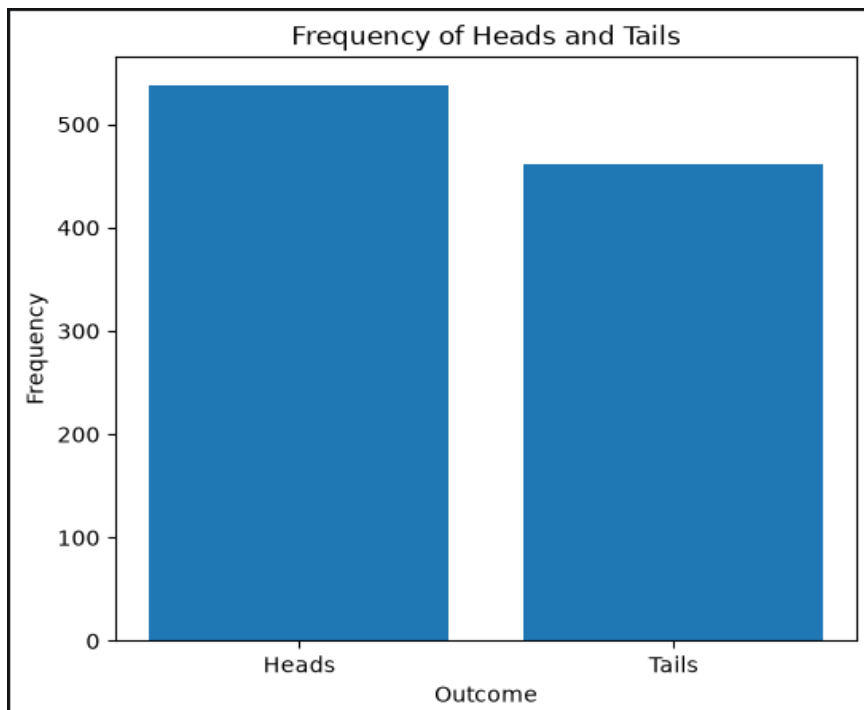
Pie Chart - 10,000 Tosses

```
plt.figure(figsize=(6, 6))
plt.pie(
    [heads_10000, tails_10000],
    labels=["Heads", "Tails"],
    autopct='%1.1f%%'
)
plt.title("10,000 Coin Tosses - Percentage Distribution")
plt.show()
print("KHUSHI T032")
print("\n===== PRACTICAL COMPLETED =====")
```

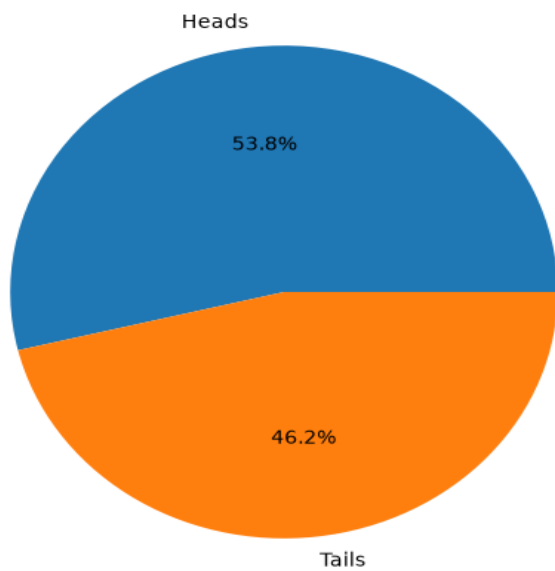
OUTPUT:-

```
Total Tosses: 1000
Heads: 538
Tails: 462
Experimental Probability of Heads: 0.538
Experimental Probability of Tails: 0.462

Theoretical Probability:
Heads: 0.5
Tails: 0.5
```



Percentage of Heads and Tails

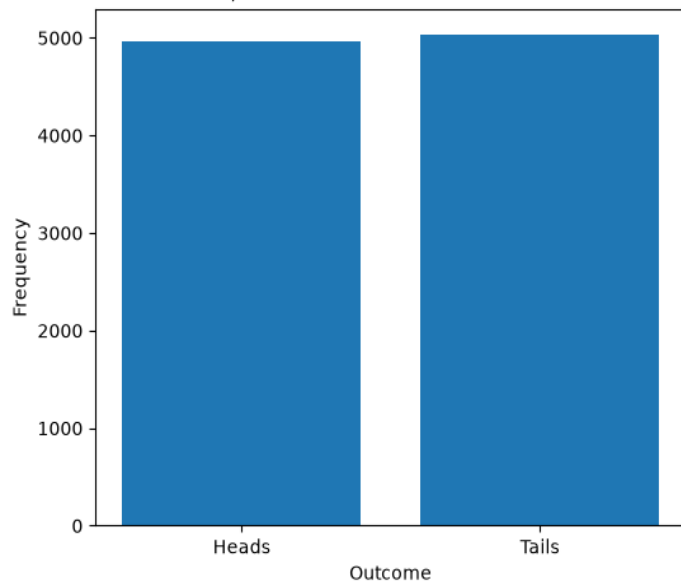


If coin is tossed 10 times and Heads occurs 8 times:
Experimental Probability of Heads: 0.8

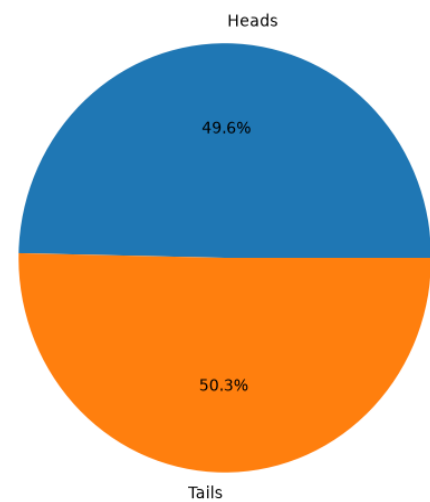
----- 10,000 Coin Toss Analysis -----

Total Tosses: 10000
Number of Heads: 4965
Number of Tails: 5035
Experimental Probability of Heads: 0.4965
Experimental Probability of Tails: 0.5035
Theoretical Probability of Heads: 0.5
Theoretical Probability of Tails: 0.5
Difference for Heads: 0.00350000000000003
Difference for Tails: 0.0034999999999999476

10,000 Coin Tosses - Heads vs Tails



10,000 Coin Tosses - Percentage Distribution



KHUSHI T032

===== PRACTICAL COMPLETED =====